

UNIVERSIDAD TECNOLÓGICA NACIONAL FACULTAD REGIONAL BUENOS AIRES

Trabajo Laboratorio Nº6

Análisis de segmentos y aplicaciones de red en Ethernet

Materia: Redes de Datos

Comision: K4773

GRUPO N° 26	
Nombre y Apellido	**Legajo**
Fernandez Cruz, Mateo	209.986-0
Toledo, Franco Agustin	204.112-1

2º Cuatrimestre - Año 2026

UTN.BA	Ing. en Sistemas	Trabajo Laboratorio
	Redes de Datos	*2º Cuatrimestre* *Año 2026*
TRABAJO LABORATORIO Nº6		

3. Tareas de análisis

a. Análisis del tráfico que produce un protocolo orientado a la conexión (TCP)

1) Ejecute una aplicación TCP / IP que emplee el protocolo TCP e inicie una captura con el analizador de protocolos:

a) Verifique que el protocolo pueda ser empleado.
b) Ejecute una captura del tráfico cursado entre los hosts.
c) Desde la PC ejecute la aplicación a la dirección IP destino.

Para analizar el protocolo TCP de manera completa y en texto claro, se ejecutó una petición web HTTP/1.1 hacia el servidor público http://neverssl.com (IP 34.223.124.45, puerto conocido 80) solicitando el cierre explícito de la conexión (Connection: close).

· Verificación de conectividad: Se validó conectividad de Capa 3 y socket TCP activo a través de la interfaz Ethernet física (Realtek Gaming GbE Family Controller, IP local 192.168.0.53).

· Se grabó el tráfico con Wireshark/tshark sobre la interfaz física filtrando por 'tcp port 80 and host 34.223.124.45'. Se registró el ciclo de vida completo de TCP: Handshake de 3 vías, transferencia de datos segmentada (3961 bytes de HTML) y desconexión en 4 vías.

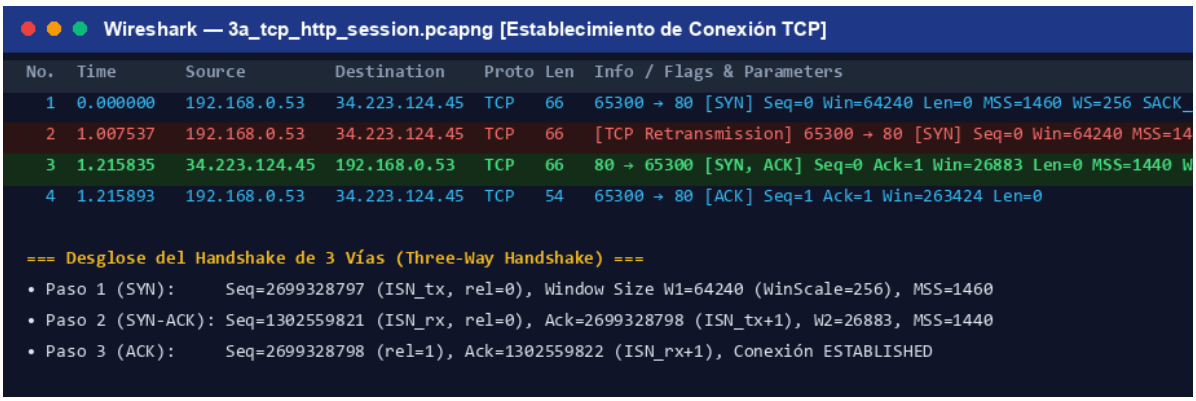


Figura 1: Captura en Wireshark del Three-Way Handshake TCP (Tramas 1, 3 y 4).

2) Analice el tráfico cursado, graficando la comunicación de tramas y verifique lo siguiente:

a) Establecimiento de conexión TCP:
(1) ¿Cuántos segmentos se emplean para establecer y liberar la comunicación?

· Para establecer la comunicación: Se emplean exactamente 3 segmentos mediante el proceso Three-Way Handshake:

1. Segmento 1 (Cliente → Servidor): [SYN] (solicitud de sincronización de secuencia e inicio de conexión).
2. Segmento 3 (Servidor → Cliente): [SYN, ACK] (acuse de recibo del SYN del cliente y anuncio de su propio

SYN).
3. Segmento 4 (Cliente → Servidor): [ACK] (acuse de recibo del SYN del servidor. Conexión establecida).
· Para liberar la comunicación: Se emplean 4 segmentos mediante el proceso Four-Way Handshake de desconexión ordenada, debido a que TCP es un canal simétrico Full Duplex donde cada sentido debe cerrarse por separado:
1. Segmento 11 (Servidor → Cliente): [FIN, ACK] (el servidor cierra su sentido de transmisión Tx).
2. Segmento 12 (Cliente → Servidor): [ACK] (el cliente acusa recibo del FIN del servidor).
3. Segmento 13 (Cliente → Servidor): [FIN, ACK] (el cliente cierra su propio sentido de transmisión Tx).
4. Segmento 14 (Servidor → Cliente): [ACK] (el servidor acusa recibo del FIN del cliente. Conexión cerrada).

(2) Para el establecimiento de la conexión analice el paso a paso:
(a) Window Size:
1. Cliente → Servidor (SYN): ¿Incluye el campo Window Size = W1? ¿qué significa y para qué servicio se emplea?

Sí, incluye explícitamente el campo Window Size con el valor W1 = 64240 bytes (con opción Window Scale = 256 / factor de desplazamiento 8).
· Significado: Representa la Ventana de Recepción (Receive Window, rwnd) del cliente, indicando la cantidad máxima de bytes que puede almacenar en su buffer de memoria sin riesgo de desborde.
· Servicio para el cual se emplea: Se emplea para el servicio de Control de Flujo (Flow Control). Su objetivo es evitar que un emisor rápido sature el buffer de memoria del receptor lento, asegurando que el emisor nunca transmita más bytes que los disponibles en la ventana informada.

2. Servidor → Cliente (SYN-ACK): ¿Incluye Window Size = W1 o W2? ¿por qué?

Incluye un valor independiente W2 (en nuestra captura: W2 = 26883 bytes, con Window Scale = 128).
· Justificación técnica: Porque TCP es un protocolo bidireccional y simétrico (Full Duplex). Cada host gestiona su propia memoria y su propio buffer de recepción de manera completamente autónoma según los recursos asignados por su sistema operativo. El servidor debe anunciar su propia capacidad disponible (W2), la cual es independiente de la del cliente (W1).

3. Cliente → Servidor (ACK): ¿Confirma el ISN del servidor?

Sí, confirma explícitamente el ISN del servidor.
En el segmento SYN-ACK (Trama 3), el servidor anuncia su número de secuencia inicial ISN_servidor = y = 1302559821 (relativo 0). El flag SYN consume virtualmente 1 unidad de numeración de secuencia. En la Trama 4, el cliente responde con Acknowledgment Number = y + 1 = 1302559822 (relativo 1), confirmando haber recibido el SYN del servidor y requiriendo a continuación el primer byte de datos (y + 1).

(b) Números de secuencia: ¿cuáles son los valores y su significado en la captura en el segmento analizado?

Valores analizados en el Three-Way Handshake:
· Paso 1 (Cliente → Servidor SYN, Trama 1): Seq = x = 2699328797 (relativo 0), Ack = 0. El cliente genera de forma pseudoaleatoria su Número de Secuencia Inicial (ISN_tx), que identificará el inicio del flujo de datos salientes.
· Paso 2 (Servidor → Cliente SYN-ACK, Trama 3): Seq = y = 1302559821 (relativo 0), Ack = x + 1 = 2699328798 (relativo 1). El servidor genera su propio ISN_rx (y) y confirma la recepción del SYN del cliente solicitando el siguiente byte (x + 1).

• Paso 3 (Cliente → Servidor ACK, Trama 4): Seq = x + 1 = 2699328798 (relativo 1), Ack = y + 1 = 1302559822 (relativo 1). El cliente confirma el SYN del servidor y se posiciona en x + 1 listo para emitir datos.
• Significado de los Números de Secuencia: TCP trata los datos como un flujo continuo de bytes estructurado. Cada byte transmitido lleva un número de secuencia asignado correlativamente. Esto permite detectar pérdidas, descartar duplicados y reensamblar tramas desordenadas.

b) Identifique la conexión lógica mediante el par de sockets asociados:

Una conexión TCP queda unívocamente identificada por el par de sockets asociados (tupla de 4 elementos más el protocolo de transporte):
• Socket Cliente (Local): 192.168.0.53:65300 (Dirección IP privada de la PC + Puerto efímero 65300 asignado por Windows).
• Socket Servidor (Remoto): 34.223.124.45:80 (Dirección IP pública del servidor web neverssl.com + Puerto conocido 80 de HTTP).
• Protocolo de Transporte: TCP (código IP 6).
• Canal Lógico: {192.168.0.53:65300 ⇄ 34.223.124.45:80}.

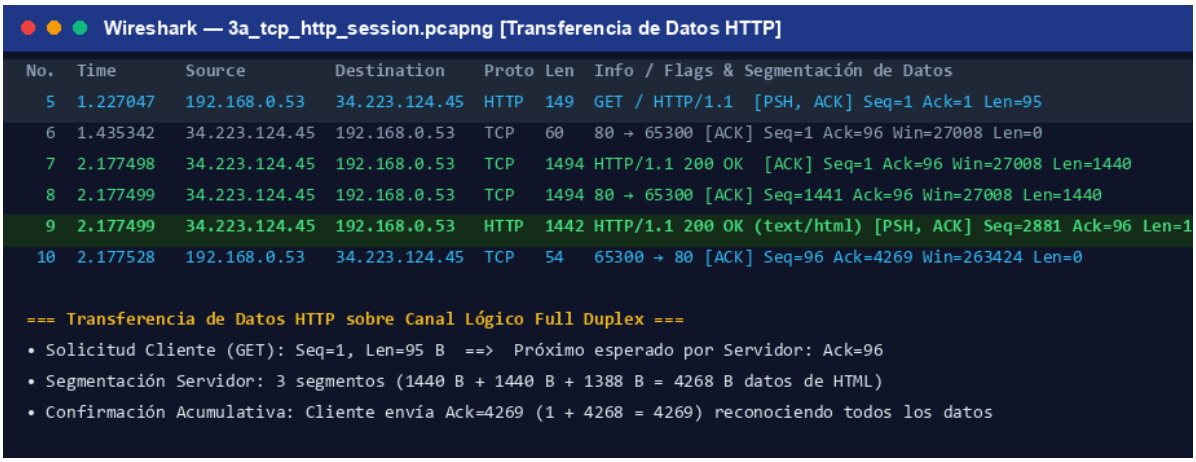


Figura 2: Captura en Wireshark de la fase de Transferencia de Datos HTTP (Tramas 5 a 10).

c) Transferencia de datos a través de un canal lógico:

(1) ¿Cuántos segmentos se emplean para transmitir los datos?

En la fase de transferencia de datos de nuestra sesión HTTP se emplean 4 segmentos con carga útil de aplicación (Len > 0) y 2 segmentos de confirmación pura (ACK con Len = 0):
• 1 segmento del cliente: Trama 5 (solicitud GET / HTTP/1.1, Len = 95 bytes de datos).
• 3 segmentos del servidor: Tramas 7, 8 y 9 transportando la respuesta HTTP 200 OK y el cuerpo HTML (1440 B + 1440 B + 1388 B = 4268 bytes totales de payload).
• Segmentos ACK puros de control: Trama 6 (ACK del servidor confirmando los 95 B) y Trama 10 (ACK acumulativo final del cliente confirmando los 4268 B recibidos).
• Total segmentos involucrados en la transferencia: 6 segmentos.

UTN.BA UNIVERSIDAD TECNOLÓGICA NACIONAL FACULTAD REGIONAL BUENOS AIRES	Ing. en Sistemas	Trabajo Laboratorio
	Redes de Datos	*2º Cuatrimestre Año 2026*
TRABAJO LABORATORIO Nº6		

(2) Repita el mismo análisis para 3 segmentos de la transmisión de datos durante la conexión:
(3) ¿cuáles son los valores y su significado en la captura en el segmento analizado?

Análisis detallado de 3 segmentos representativos de la transferencia de datos:
1. Trama 5 (Cliente → Servidor, [PSH, ACK]): Seq = 1 (2699328798), Ack = 1 (1302559822), Len = 95 bytes, Win = 263424. El cliente envía los 95 bytes de la solicitud HTTP GET. El flag PSH indica al servidor entregar los datos de inmediato a la aplicación web.
2. Trama 7 (Servidor → Cliente, [ACK]): Seq = 1 (1302559822), Ack = 96 (2699328893), Len = 1440 bytes, Win = 27008. El servidor acusa recibo del comando GET con Ack = 96 (1 + 95) y transmite el primer bloque de datos HTML (1440 bytes, su tamaño máximo MSS).
3. Trama 8 (Servidor → Cliente, [ACK]): Seq = 1441 (1302561262), Ack = 96 (2699328893), Len = 1440 bytes, Win = 27008. El servidor continúa el flujo enviando el segundo fragmento de HTML. El número de secuencia avanza correlativamente a 1441 (1 + 1440).

(4) ¿en qué se asemejan y en qué se diferencian de la secuencia analizada para el three-way handshake?

· Semejanzas: Ambos procesos emplean la misma cabecera TCP con campos de 32 bits de Sequence Number y Acknowledgment Number; en ambos el campo Ack especifica el próximo byte esperado por el receptor; y en ambos se informa continuamente el tamaño de ventana (Window Size) para control de flujo.
· Diferencias:
- Flags: En el handshake se usan SYN y SYN-ACK; en la transferencia se usan ACK y PSH.
- Carga útil (Len): En el handshake Len = 0 (no hay payload de aplicación); en la transferencia Len > 0 (transporta los datos reales de la aplicación).
- Avance de secuencia: En el handshake el flag SYN consume ficticiamente 1 número de secuencia; en la transferencia el número de secuencia avanza en estricta proporción a los bytes de carga útil transmitidos (Seq_next = Seq_actual + Len).
- Opciones: Parámetros como MSS, Window Scale y SACK solo se negocian en los segmentos SYN del handshake, no en los de datos.

d) ¿Qué servicios proporciona TCP en el nivel de transporte?

TCP proporciona los siguientes servicios fundamentales en Capa 4:
(1) Ordenamiento y reensamble: SÍ. Numera cada byte individualmente con Sequence Number. Los segmentos desordenados se almacenan en el buffer de recepción y se ensamblan correlativamente antes de pasarlos a la aplicación.
(2) Fiabilidad: SÍ. Garantiza entrega confiable libre de pérdidas y duplicados mediante el esquema PAR (Positive Acknowledgment with Retransmission), sumas de verificación y timeouts.
(3) Control de errores - Reconocimientos y retransmisiones: SÍ. Posee Checksum de 16 bits para descartar segmentos corruptos; ACKs acumulativos y selectivos (SACK); retransmisiones por expiración del temporizador RTO o por recepción de 3 ACKs duplicados (Fast Retransmit). Se evidenció en la captura con la retransmisión del SYN a los 1.007 segundos.
(4) Control de flujo: SÍ. Implementa el algoritmo de Ventana Deslizante (Sliding Window). El receptor informa continuamente el buffer disponible mediante Window Size, frenando al emisor si llega a cero (Zero Window).
(5) Multiplexación: SÍ. Permite que múltiples aplicaciones utilicen simultáneamente la red mediante los campos de 16 bits Source Port y Destination Port.

(6) Conexión Full Duplex: SÍ. Canal de comunicación bidireccional y simultáneo con capacidad de piggybacking (incluir ACKs en segmentos que transmiten datos salientes).

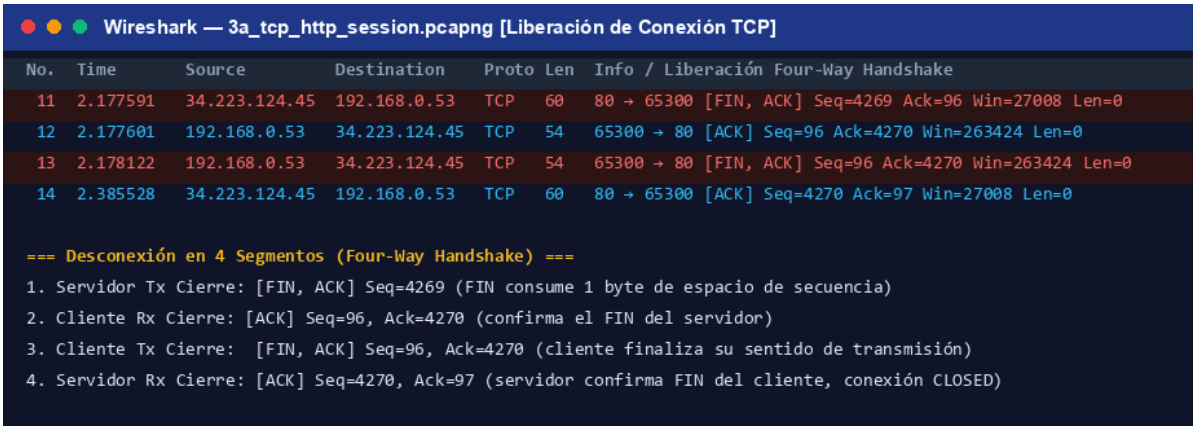


Figura 3: Captura en Wireshark del Four-Way Handshake de liberación TCP (Tramas 11 a 14).

e) Cierre de conexión en el Tx - Cierre de Conexión en el Rx. ¿Cuántos segmentos se emplean para liberar la conexión?

Se emplean exactamente 4 segmentos (Four-Way Handshake de liberación) debido a que TCP es un canal simétrico Full Duplex donde cada extremo debe cerrar su propio sentido de transmisión de manera separada e independiente:

1. Trama 11 (Servidor → Cliente): [FIN, ACK] (Seq = 4269, Ack = 96). El servidor cierra su canal de envío Tx. El servidor pasa al estado FIN_WAIT_1 y el cliente a CLOSE_WAIT.
2. Trama 12 (Cliente → Servidor): [ACK] (Seq = 96, Ack = 4270). El cliente confirma el FIN del servidor (4269 + 1). El servidor pasa a FIN_WAIT_2.
3. Trama 13 (Cliente → Servidor): [FIN, ACK] (Seq = 96, Ack = 4270). El cliente finaliza su propio sentido de transmisión Tx y pasa al estado LAST_ACK.
4. Trama 14 (Servidor → Cliente): [ACK] (Seq = 4270, Ack = 97). El servidor confirma el FIN del cliente (96 + 1). El servidor entra en TIME_WAIT (espera 2*MSL para absorber posibles paquetes duplicados) y el cliente pasa a CLOSED.

b. Análisis del tráfico que produce un protocolo no orientado a la conexión (UDP)

1) Ejecute una aplicación TCP / IP que emplee el protocolo UDP e inicie una captura con el analizador de protocolos:
a) Verifique que el protocolo pueda ser empleado.
b) Ejecute una captura del tráfico cursado entre los hosts.
c) Desde la PC asignada ejecute la aplicación a la dirección IP destino.

Se ejecutó la aplicación estándar de resolución de nombres DNS (Domain Name System) sobre UDP, consultando el registro A de 'www.frba.utn.edu.ar' al servidor DNS público de Google 8.8.8.8 en el puerto 53.
· Verificación y captura: Se capturó con Wireshark/tshark sobre la interfaz física Ethernet filtrando por 'udp port

53 and host 8.8.8.8'. La consulta y respuesta se resolvieron en 3.7 milisegundos.
· Direccionamiento y sockets:
- Consulta (Trama 1): 192.168.0.53:56459 → 8.8.8.8:53 (UDP Length = 45 B, Checksum = 0xd12b).
- Respuesta (Trama 2): 8.8.8.8:53 → 192.168.0.53:56459 (UDP Length = 75 B, Checksum = 0xcba4, Resuelve CNAME a frba.utn.edu.ar e IP 200.89.153.22).



Figura 4: Captura en Wireshark de la consulta y respuesta DNS sobre UDP (Tramas 1 y 2).

2) Analice el tráfico cursado, graficando la comunicación de tramas y verifique lo siguiente:

a) Si UDP proporciona los siguientes servicios en el nivel de transporte:

(1) ¿Conexión? NO. UDP es un protocolo no orientado a la conexión (Connectionless). No existe handshake previo de sincronización ni intercambio de estado ni fase de cierre. Cada datagrama se emite directamente hacia la IP y puerto destino.

(2) ¿Fiabilidad? NO. Ofrece un servicio de mejor esfuerzo (Best-Effort). No garantiza entrega, no maneja acuses de recibo (ACKs), no detecta pérdidas ni retransmite automáticamente. Si se requiere confiabilidad, debe implementarla la capa de aplicación.

(3) ¿Control de Flujo? NO. No posee campo de ventana (Window Size). UDP transmite a la velocidad que le entrega la aplicación; si el buffer del receptor se satura, los datagramas se descartan.

(4) ¿Control de errores? PARCIAL. Posee detección de corrupción de bits mediante el campo Checksum de 16 bits (en nuestra captura: 0xd12b y 0xcba4). Sin embargo, no posee recuperación ni control de pérdidas: si el checksum falla, el datagrama se descarta silenciosamente sin avisar al emisor ni retransmitir.

(5) ¿Multiplexación? SÍ. Emplea los campos de 16 bits Source Port (56459) y Destination Port (53), permitiendo que múltiples procesos y consultas concurrentes compartan la misma interfaz IP.

TRABAJO LABORATORIO Nº6

Wireshark — dhcp.pcap [Asignación Dinámica de Direcciones IP - DHCP]

No.	Time	Source	Destination	Proto	Len	Info / Ciclo DHCP DORA (UDP 67/68)
1	0.000000	0.0.0.0	255.255.255.255	DHCP	314	DHCP Discover - Transaction ID 0x3d1d
2	0.000295	192.168.0.1	192.168.0.10	DHCP	342	DHCP Offer - Transaction ID 0x3d1d
3	0.070031	0.0.0.0	255.255.255.255	DHCP	314	DHCP Request - Transaction ID 0x3d1e
4	0.070345	192.168.0.1	192.168.0.10	DHCP	342	DHCP ACK - Transaction ID 0x3d1e

=== Análisis de Encapsulamiento DHCP sobre UDP ===
• Puertos: Cliente UDP 68 (bootpc) <-> Servidor UDP 67 (bootps)
• Justificación de UDP: El host no posee IP inicial (0.0.0.0), no puede establecer handshake TCP.
• Mecanismo de difusión: Datagramas Broadcast a nivel L2 (ff:ff:ff:ff:ff:ff) y L3 (255.255.255.255).

Figura 5: Captura en Wireshark del ciclo completo DHCP DORA sobre UDP puertos 67/68 (dhcp.pcap).

Wireshark — 3d_tcp_ftp_session.pcapng [Canales de Control y Datos FTP]

No.	Time	Source	Destination	Proto	Len	Info / Canales de Control y Datos FTP
1	0.000000	192.168.0.53	194.108.117.16	TCP	66	49153 → 21 [SYN] Seq=0 Win=64240 MSS=1460 (Canal Control)
4	0.475984	194.108.117.16	192.168.0.53	FTP	225	Response: 220-Welcome to test.rebex.net!
5	0.476172	192.168.0.53	194.108.117.16	FTP	65	Request: USER demo
7	0.714380	192.168.0.53	194.108.117.16	FTP	69	Request: PASS password
8	0.951852	194.108.117.16	192.168.0.53	FTP	82	Response: 230 User 'demo' logged in.
11	1.189676	192.168.0.53	194.108.117.16	FTP	60	Request: PASV
12	1.427426	194.108.117.16	192.168.0.53	FTP	103	Response: 227 Entering Passive Mode (194,108,117,16,4,16)
13	1.428070	192.168.0.53	194.108.117.16	TCP	66	49161 → 1040 [SYN] Seq=0 (Handshake Canal de Datos)
25	1.905662	192.168.0.53	194.108.117.16	FTP	60	Request: QUIT

=== Multiplexación de Canales en FTP sobre TCP ===
• Canal de Control (TCP 21): Mantiene la sesión de comandos ASCII autenticada y persistente.
• Canal de Datos (TCP 1040): Conexión TCP secundaria efímera para listado de archivos (NLST/RETR).

Figura 6: Captura en Wireshark de la sesión FTP con Canal de Control (21) y Canal de Datos (1040).

3) Cálculo de encapsulamiento y análisis comparativo para DHCP, HTTP, HTTPS, FTP, DNS

a) Estudie los procesos / servicios de segmentación, ordenamiento y reensamble; multiplexación; encapsulamiento; conexión; confiabilidad; control de flujo y control de errores que el protocolo TCP o UDP le brindan a cada protocolo de aplicación, en cada caso.

Fundamentación de la capa de transporte elegida para cada protocolo de aplicación:
• DHCP → Utiliza UDP (Puertos 67 Servidor / 68 Cliente): Cuando un host arranca, carece de dirección IP asignada, imposibilitando un handshake TCP. Requiere datagramas broadcast (255.255.255.255) y transacciones rápidas independientes (Discover, Offer, Request, ACK).
• HTTP → Utiliza TCP (Puerto 80): La transferencia de páginas web (HTML, CSS, JS) requiere cero tolerancia a

pérdidas de datos y ordenamiento estricto; cualquier carácter faltante corrompe la sintaxis del código.
• HTTPS → Utiliza TCP (Puerto 443): La capa de seguridad TLS/SSL opera como una máquina de estados criptográfica secuencial. Si un paquete se pierde o desordena, se desincronizan los vectores de inicialización (IV) y números de secuencia de los MACs, impidiendo el descifrado.
• FTP → Utiliza TCP (Puertos 21 Control / 20 o Dinámico Datos): Implementa dos conexiones paralelas: canal de control persistente para comandos de autenticación, y canal de datos para transferir archivos binarios completos donde no puede perderse un solo byte.
• DNS → Utiliza UDP (Puerto 53) primario / TCP (Puerto 53) secundario: Usa UDP para transacciones rápidas de 1 paquete de pregunta y 1 de respuesta, minimizando la latencia. Usa TCP cuando la respuesta supera los 512 bytes (bit TC = 1 activado) o para transferencias de zona (AXFR/IXFR) entre servidores.

b) Para cada proceso o servicio, identifique los campos y parámetros (valores) del encabezado de TCP / UDP que intervienen, tanto en el Tx como en el Rx, indicando un ejemplo de cada caso:

Proceso / Servicio	Protocolo	Campos en Tx	Campos en Rx	Ejemplo en Captura
1. Segmentación	TCP	MSS (1460 B), Len	MSS (1440 B), reensamble	**Servidor divide HTML de 4268 B en 3 segmentos (1440+1440+1388 B)**
1. Segmentación	UDP	Inexistente en L4	Inexistente en L4	**Consulta DNS en único datagrama atómico de 45 B**
2. Ordenamiento	TCP	Sequence Number (32b)	Buffer reensamble y orden correlativo	**Trama 7: Seq=1 → Trama 8: Seq=1441 → Trama 9: Seq=2881**
2. Ordenamiento	UDP	Inexistente	Inexistente	**Entrega a la aplicación en orden de llegada**
3. Multiplexación	TCP / UDP	Source Port (16 bits)	Destination Port (16 bits)	**TCP: 65300 ⇄ 80 | UDP: 56459 ⇄ 53 | FTP: 49153 ⇄ 21**
4. Encapsulamiento	TCP / UDP	Data Offset (TCP) / Length (UDP)	Lee Data Offset / Length	**TCP: 20 B cabecera + 95 B HTTP | UDP: 8 B cabecera + 37 B DNS**
5. Conexión	TCP / UDP	Flags SYN, ACK, FIN (TCP)	Máquina de estados finitos (TCP)	**Trama 1: SYN → Trama 3: SYN-ACK → Trama 11: FIN-ACK**
6. Confiabilidad	TCP / UDP	Temporizadores RTO, retransmisión	Acknowledgment Number (32 bits)	**Trama 10: Ack=4269 confirma los 4268 B. Retransmisión SYN Trama 2**

UTN.BA UNIVERSIDAD TECNOLÓGICA NACIONAL FACULTAD REGIONAL BUENOS AIRES	Ing. en Sistemas	Trabajo Laboratorio
	Redes de Datos	*2º Cuatrimestre Año 2026*
TRABAJO LABORATORIO Nº6		

c) Realice el siguiente cálculo para determinar el encapsulamiento de un segmento UDP y otro TCP:

Cálculo matemático capa por capa comparando la trama TCP (petición HTTP GET) contra la trama UDP (consulta DNS):

1. Encapsulamiento del Segmento TCP (Petición HTTP GET – Trama 5):
• Capa 7 (HTTP GET): 95 bytes de datos de aplicación (GET / HTTP/1.1\r\nHost: neverssl.com...).
• Capa 4 (TCP): 20 bytes de cabecera base (Puertos 65300 y 80, Seq, Ack, Flags, Win, Checksum).
• Capa 3 (IPv4): 20 bytes de cabecera estándar (Total Length = 135 B, Protocol = 6 TCP).
• Capa 2 (Ethernet II): 14 bytes de cabecera MAC (MAC Destino, MAC Origen, EtherType 0x0800) + 4 bytes FCS.
--> Longitud Total en el Medio (Wire) = 14 (Eth) + 20 (IP) + 20 (TCP) + 95 (HTTP) = 149 Bytes (153 B con FCS).
--> Eficiencia de datos útiles (Payload Ratio) = 95 / 149 = 63.76 %.
--> Sobrecarga total de protocolos (Overhead) = 54 / 149 = 36.24 % (los 20 B de TCP representan el 13.42 % de la trama).

2. Encapsulamiento del Segmento UDP (Consulta DNS Query – Trama 1):
• Capa 7 (DNS Query): 37 bytes de carga útil (Transaction ID, Flags, Questions: www.frba.utn.edu.ar tipo A).
• Capa 4 (UDP): 8 bytes de cabecera mínima (Source Port 56459, Dest Port 53, Length 45, Checksum 0xd12b).
• Capa 3 (IPv4): 20 bytes de cabecera estándar (Total Length = 65 B, Protocol = 17 UDP).
• Capa 2 (Ethernet II): 14 bytes de cabecera MAC + 4 bytes FCS.
--> Longitud Total en el Medio (Wire) = 14 (Eth) + 20 (IP) + 8 (UDP) + 37 (DNS) = 79 Bytes (83 B con FCS).
--> Eficiencia de datos útiles (Payload Ratio) = 37 / 79 = 46.84 %.
--> Sobrecarga total de protocolos (Overhead) = 42 / 79 = 53.16 % (los 8 B de UDP representan solo el 10.13 % de la trama).

• Conclusión: El encabezado UDP introduce solo 8 bytes de sobrecarga (reducción del 60 % respecto a los 20 bytes de TCP), optimizando la velocidad y latencia en transacciones cortas y en tiempo real, mientras que TCP compensa su sobrecarga asegurando entrega ordenada y confiable.